NAME: K. SWARNA VARSHINI

REG.NO: 192324081

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1715

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Duration: 1 Hour**

Total Marks:50

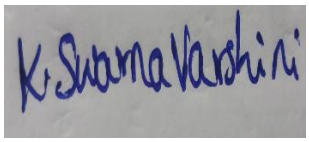
Annexure A

Analytical Problem Solving

Code of Conduct:

I _K. Swarna Varshini__(192324081)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent?

ii. Characterize the operating environment.

iii. What are the actions the agent can take?

iv. How can one evaluate the performance of the agent?

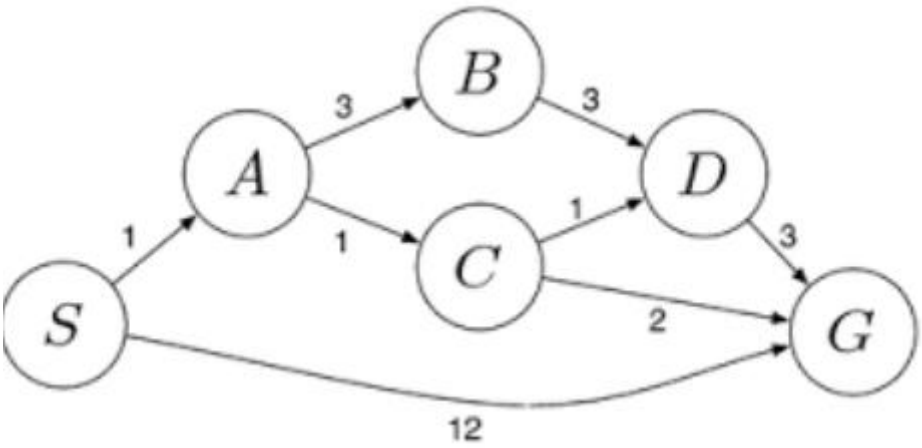
v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Problem Understanding	20% (2)	/2	/2	/2	/2	/2	/10
Method Selection	20% (2)	/2	/2	/2	/2	/2	/10
Solution Process	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Accuracy of Calculation	20% (2)	/2	/2	/2	/2	/2	/10
Interpretation of Results	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Total per Question	**10 Marks**	**/10**	**/10**	**/10**	**/10**	**/10**	**/50**

100

ANSWERS

1. Problem Statement

The Water Jug Problem is a classic Artificial Intelligence search problem. We are given two jugs of capacities **4 gallons** and **3 gallons**. There are no measurement markings on the jugs. The objective is to obtain exactly **2 gallons of water in the 4-gallon jug** using only the following operations:

- Fill a jug completely from the pump.
- Empty a jug completely onto the ground.
- Pour water from one jug to another until one jug becomes empty or the other becomes full.

The problem is solved using **State Space Search**, where each state is represented as **(4-gallon jug, 3-gallon jug)**.

State Representation

A state is represented as **(x, y)**

where

- x = amount of water in the 4-gallon jug
- y = amount of water in the 3-gallon

jug Initial State: (0,0)

Goal State: (2,0)

Step	4-Gallon Jug	3-Gallon Jug	Action
Initial	0	0	Start
1	0	3	Fill 3-gallon jug
2	3	0	Pour into 4-gallon jug
3	3	3	Fill 3-gallon jug again

4	4	2	Pour into 4-gallon until full
5	0	2	Empty 4-gallon jug
6	2	0	Pour remaining 2 gallons into 4-gallon jug

Goal Achieved: 4-gallon jug = 2 gallons

Pseudocode

Start

Initialize state = (0,0)
Repeat
If 3-gallon jug is empty
  Fill 3-gallon jug
Else if 4-gallon jug is not full

  Pour water from 3-gallon to 4-gallon
Else
  Empty 4-gallon jug
Repeat until state =
(2,0) Stop

Conclusion

The Water Jug problem demonstrates **state-space search** in Artificial Intelligence. By applying legal operations systematically, the goal state **(2,0)** is reached successfully.

2. Problem Statement

A Mars Rover is an autonomous intelligent agent that explores the Martian surface. Since direct human control is impossible because of communication delays, the rover must perceive its environment, make intelligent decisions, navigate safely, collect samples, and send scientific information back to Earth.

- ### Percepts

Percepts are the information received by the rover through its sensors. The rover perceives:

- Images from navigation cameras
- Rock and soil characteristics

- Terrain information
- Temperature
- Atmospheric pressure
- Wind speed
- Chemical composition of rocks
- GPS-like localization data
- Obstacle locations
- Battery level

- Operating Environment

The Mars Rover works in a **partially observable**, **dynamic**, **sequential**, **stochastic**, **continuous**, and **single-agent** environment.

Characteristics

- Partially observable because it cannot observe the whole environment.
- Dynamic because weather and terrain change.
- Sequential because one action affects future actions.
- Stochastic because wheel slippage and unexpected obstacles may occur.
- Continuous because movement is continuous.
- Single agent because only one rover performs exploration.

- Actions

The rover can

- Move forward
- Move backward
- Turn left
- Turn right

- Capture images
- Collect rock samples
- Drill rocks
- Analyze soil
- Avoid obstacles
- Send data to Earth
- Recharge batteries using solar panels
- Stop movement
- Performance Measure

The performance of the rover is evaluated using:

- Number of successful samples collected
- Accuracy of scientific observations
- Safe navigation
- Distance covered

- Battery efficiency
- Number of obstacles avoided
- Communication success
- Mission completion time

A better-performing rover collects maximum scientific data while consuming minimum energy.

- Suitable Agent Architecture

The most suitable architecture is a **Learning Agent**.

Components

- Performance element
- Learning element
- Critic

- Problem generator

Reason

A learning agent improves its performance from previous experiences. It adapts to new terrain, avoids obstacles intelligently, and makes better navigation decisions over time.

Conclusion

The Mars Rover is an excellent example of a **Learning Agent** because it continuously improves its exploration capability using environmental feedback.

3. Problem Statement

The objective is to place **8 queens** on an **8 × 8 chessboard** such that **no queen attacks another queen**. Queens attack along

- Rows
- Columns
- Diagonals

The problem is solved using **State Space Search** or **Backtracking Search**.

Problem Formulation

Initial State Diagram

Empty chessboard.

	C1	C2	C3	C4	C5	C6	C7	C8
R1								
R2								
R3								
R4								
R5								
R6								
R7								
R8								

Queen Attack Diagram

↖ ↑ ↗

\|/

← ← ← Q → → →

/|\

↙ ↓ ↘

A queen attacks in:

- Horizontal direction
- Vertical direction
- Both diagonals

Therefore, no two queens should lie in these directions.

Example of a Wrong Configuration

	C1	C2	C3	C4	C5	C6	C7	C8
R1	Q							Q
R2								
R3		Q						
R4				Q				
R5						Q		
R6								
R7			Q					
R8					Q			

Queen in **C1** and **C8** are on the same diagonal, so this configuration is invalid.

Goal State (Correct 8-Queens Solution)

	C1	C2	C3	C4	C5	C6	C7	C8
R1	Q							
R2					Q			
R3								Q
R4						Q		
R5			Q					
R6							Q	
R7		Q						
R8				Q				

Queen positions are:

- (R1, C1)
- (R2, C5)
- (R3, C8)
- (R4, C6)
- (R5, C3)
- (R6, C7)
- (R7, C2)
- (R8, C4)

No two queens attack each other.

Pseudocode

PlaceQueen(column)
If column > 8
  Print solution
  Return
For each row from 1 to 8 If position is safe
Place queen

PlaceQueen (column +

1) Remove queen

End

Conclusion

The 8-Queens problem is a classic AI constraint satisfaction problem solved efficiently using Backtracking Search.

4. Problem Statement

A customer wants to travel from one location to another using the OLA Cab application. The system provides different cab types such as Micro, Mini, Sedan, Prime, and Shared. The customer selects a suitable cab based on comfort, availability, fare, and travel preferences.

Suitable Problem-Solving Agent

A **Goal-Based Agent** is most suitable.

Reason

The customer's goal is to reach the destination safely and efficiently. The agent evaluates different options and selects the one that satisfies the user's goal.

Working

1. Customer enters pickup location.
2. Customer enters destination.
3. System checks available cabs.
4. Calculates fare and estimated arrival time.
5. Displays available cab types.
6. Customer selects preferred cab.
7. Booking is confirmed.
8. Driver is assigned.
9. Customer reaches destination.

Pseudocode

Start

Input pickup

location Input destination Search available cabs Display cab types Customer selects cab Calculate fare Confirm booking

Assign nearest driver

Track ride

Reach destination

End

Conclusion

The OLA Cab application uses a **Goal-Based Agent** because it searches for the best sequence of actions to achieve the customer's travel objective.

5. Problem Statement

A logistics company operates a delivery network where packages must be transported from the starting warehouse **S** to the destination warehouse **G** with the **minimum travel cost**. Each route between warehouses has an associated travel cost. The objective is to find the **least-cost path** using the **Uniform Cost Search (UCS)** algorithm.

Edge	Cost
S → A	1
S → G	12

A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

Algorithm Steps

1. Start from source node S.
2. Insert S into the priority queue.
3. Remove the node having the smallest path cost.
4. Expand its neighboring nodes.
5. Update cumulative costs.
6. Continue until destination G is reached.
7. Return the least-cost path.

Pseudocode

```
UniformCostSearch (Start, Goal) Create priority queue
Insert Start with cost 0
While queue is not empty
Remove node with minimum cost If node is Goal
Return path
Expand neighbors
If new path cost is smaller Update priority queue End While
```

Step-by-Step UCS Execution

Step 1

```
Start Node
OPEN = [(S,0)]
CLOSED = {}
```

Expand **S**
Possible paths:

S → A = 1
S → G = 12 **OPEN**
A(1)

G(12)

Step 2

Expand **A** (Lowest Cost = 1)
New paths
S → A → B
Cost = 1+3 = 4
S → A → C
Cost = 1+1 = 2
OPEN
C(2)

B(4)

G(12)

Step 3

Expand **C** (Lowest Cost = 2)
Possible paths
S → A → C → D Cost = 2+1 = 3
S → A → C → G Cost = 2+2 = 4 OPEN
D(3)

B(4)

G(4)

G(12)

Step 4

Expand **D** (Lowest Cost = 3)

New path

S → A → C → D → G
Cost = 3+3 = 6
OPEN
B(4)

G(4)

G(6)

G(12)

Step 5

The smallest-cost goal node is
G(4)
Since UCS stops when the **goal with the minimum cost** is removed from the priority queue, the search terminates.

Least-Cost Path S → A → C → G **Total Cost**

1 + 1 + 2 = 4

Optimal Solution

S

↓
A

↓
C

↓
G

Minimum Cost = 4
Advantages

- Finds the optimal path.
- Works with different edge costs.
- Complete and optimal for positive path costs.

Disadvantages

- Uses more memory.
- Slower than BFS when the graph is very large

Conclusion

Using Uniform Cost Search, the logistics company finds the least-cost delivery route by always expanding the node with the lowest cumulative cost. For the given graph, the optimal path is:

S → A → C → G

with a minimum total cost of 4. This demonstrates how UCS efficiently identifies the most economical path in a weighted graph.